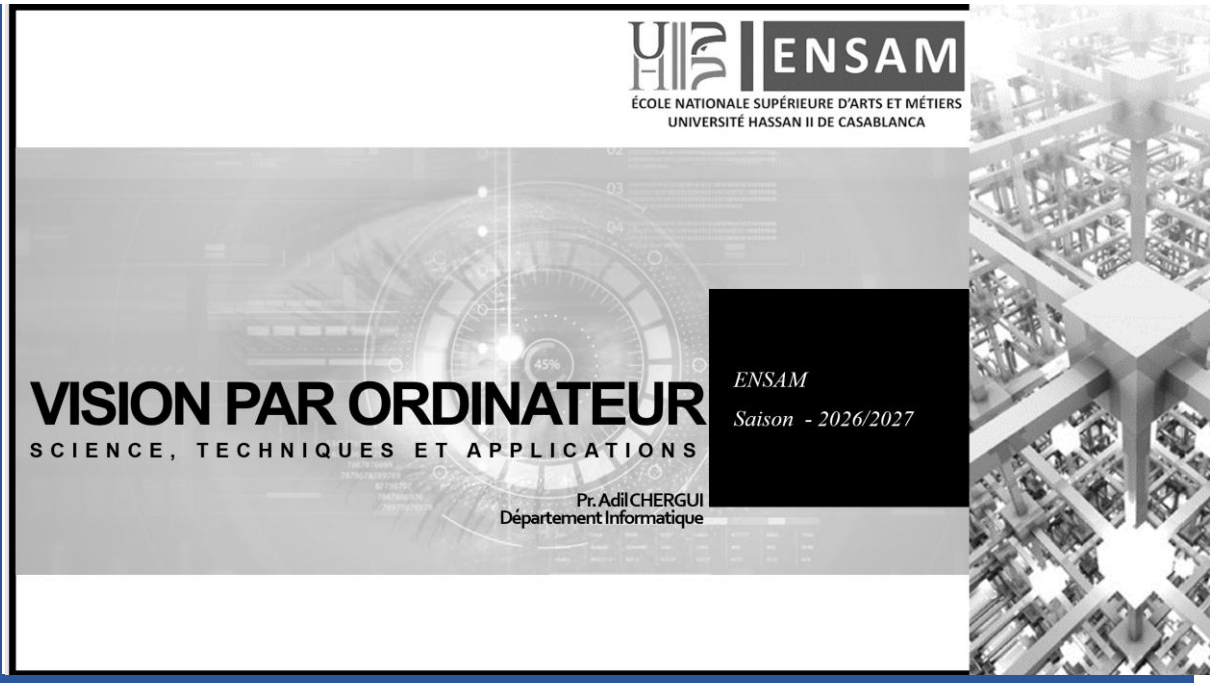


TP 8 :
Les Points D'intérêt et Descripteurs Python + OPENCV

Objectifs des TPs

Comprendre le principe du filtrage et son utilisation tout en préparant une image à des traitements tels que la détection de contours.

Compétences visées :

- Comprendre le principe de convolution ;
- Construire/utiliser un noyau de convolution ;
- Modifier les paramètres des kernels;
- Analyser l'effet du filtrage sur les détails ;
- Comparer plusieurs algorithme entre eux.

1. TP 1 : Introduction aux points d'intérêt

1.1. Objectif :

Comprendre ce qu'est un point d'intérêt (keypoint).

Un point d'intérêt est une région particulièrement informative d'une image : **coin, texture, intersection de contours**, etc.

Utiliser une image contenant des objets/textures et identifier visuellement les régions susceptibles de constituer des points d'intérêt.

Comparer :

- Régions uniformes ;
- Contours ;
- Coins.

1.2. Exemple d'image synthétique

Créer une image synthétique :

Code : exercice 1\exo1.py

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = np.zeros((500,500), dtype=np.uint8)

cv2.rectangle(image, (100,100), (400,400), 255, 3)
cv2.line(image, (100,250), (400,250), 255, 3)
cv2.circle(image, (250,250), 50, 255, 3)

plt.imshow(image, cmap="gray")
plt.show()
```

1.3. Execution

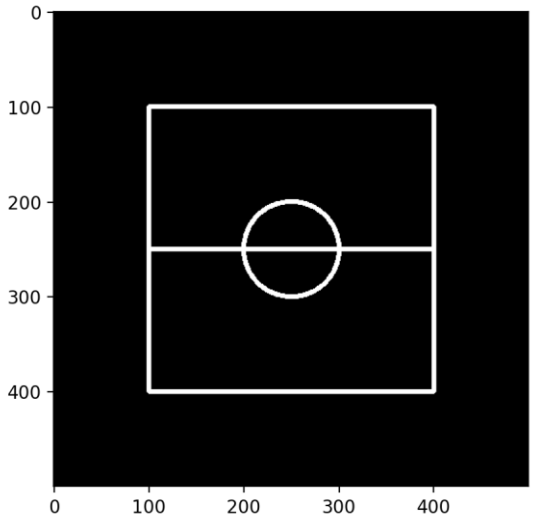


Figura 1 : Execution exo1.py

1.4. Reflexion

- Quel intérêt peut être manifester par une région uniforme ?
- Pourquoi un coin est-il généralement plus intéressant qu'une région uniforme ?
- Pourquoi un simple contour peut-il être ambigu dans une direction ?
- Comparer les quantités de ces informations

2. TP 2 : Harris Corner Detector

2.1. Objectif :

La fonction cv2.cornerHarris de la bibliothèque OpenCV détecte les coins d'une image en analysant les variations d'intensité des pixels dans toutes les directions.

.

2.2. Implémentation

Code : exercice 2\exo2.py

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("synthetic.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

gray = np.float32(gray)

corners = cv2.cornerHarris(
    gray,
    blockSize=2,
    ksize=3,
    k=0.04
)

corners = cv2.dilate(corners, None)

result = image.copy()

result[corners > 0.01 * corners.max()] = [0, 0, 255]

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

2.3. Explication sur le fonctionnement

Fonction de détection de points

La fonction **`cv2.cornerHarris`** permet de détecter les points d'intérêts.

Les paramètres de notre exemple :

```
corners = cv2.cornerHarris(
    gray,
    blockSize=2,
    ksize=3,
    k=0.04
)
```

Fonctionnent comme suite :

- **gray** : L'image d'entrée en niveaux de gris et convertie au format de type float32.
- **blockSize = 2** : La taille du voisinage (en pixels) considéré autour de chaque pixel pour l'analyse des gradients.
- **ksize = 3** : La taille de l'ouverture pour l'opérateur de Sobel utilisé lors du calcul des dérivées (doit être impaire).
- **k = 0.04** : Le paramètre libre de l'algorithme de Harris dans l'équation du score R (généralement compris entre 0.04 et 0.06)

Marquage des points :

Pour marquer visuellement en rouge les coins détectés sur l'image d'origine, nous utilisons la ligne suivante :

```
result[corners > 0.01 * corners.max()] = [0, 0, 255]
```

Son fonctionnement :

- **corners.max()** : Trouve le score le plus élevé parmi tous les coins détectés dans la matrice corners.
- **0,01*corners.max()** : Calcule un seuil de tolérance égal à 1 % du score maximum. Vous pouvez ajuster ce pourcentage (ex: 0.05 pour 5 %) pour filtrer les faux positifs et ne garder que les coins les plus nets.
- **corners > ..**. : Crée un masque booléen (une grille de True et False). Tous les pixels dont le score est supérieur au seuil passent à True.
- **result[...] = [0, 0, 255]** : Applique la couleur rouge aux pixels sélectionnés par le masque. Dans OpenCV, le format de couleur par défaut est le BGR (Blue, Green, Red). Le triplet [0, 0, 255] correspond donc à un bleu à 0, un vert à 0 et un rouge au maximum.

Ce que retourne cette fonction :

Elle renvoie une matrice de **scores** de même dimension que l'image originale, où chaque valeur représente le score de "**jonction ou coin**" (R) du pixel correspondant. Un score élevé indique un coin.

2.4. Exécution

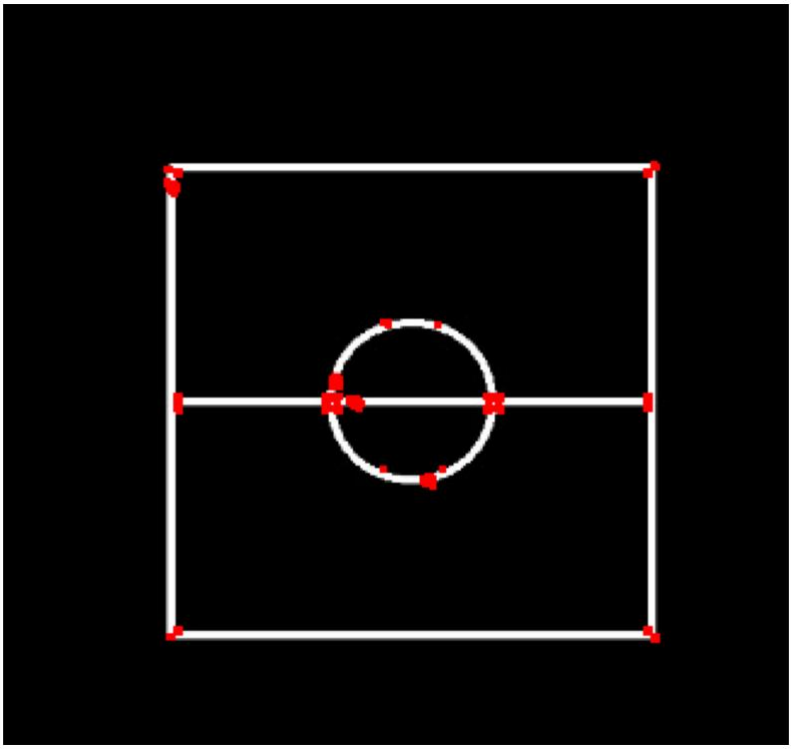


Figura 2 : Les coins de Harris sur l'image synthetique

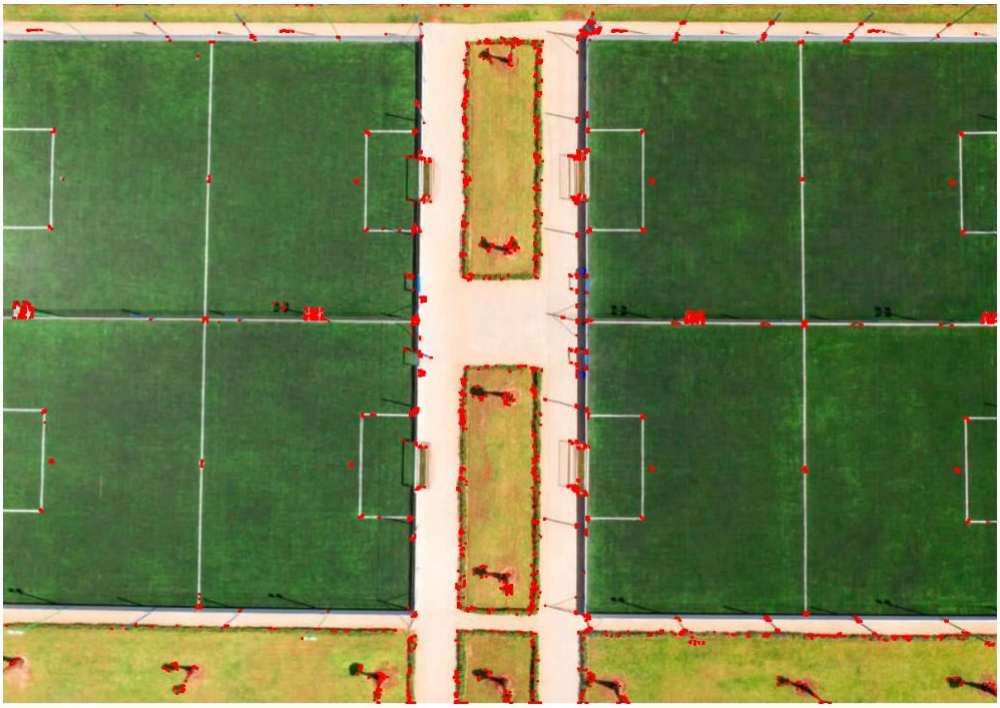


Figura 3 : Les coins de Harris detectés sur une image réele

2.5. Question à la base de tests

En faisant plusieurs tests et interprétations, produire les effets et répondre à ces questions :

Questions

1. Quel est le rôle de **blockSize** ?
2. Quel est le rôle de **ksize** ?
3. Quel est le rôle de **k** ?
4. Pourquoi utilise-t-on **dilate** ?

3. TP 3 : Shi-Tomasi / Good Features to Track

3.1. Objectif :

Utiliser Shi-Tomasi pour sélectionner les meilleurs points d'intérêt.

3.2. Implementation

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("terrain.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

corners = cv2.goodFeaturesToTrack(
    gray,
    maxCorners=100,
    qualityLevel=0.01,
    minDistance=10
)

corners = corners.astype(int)

result = image.copy()

for corner in corners:
    x, y = corner.ravel()

    cv2.circle(
        result,
        (x, y),
        4,
        (0, 0, 255),
        -1
```

```
    )

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

3.3. Explication et fonctionnement

La fonction **cv2.goodFeaturesToTrack** implémente l'algorithme de détection de coins de **Shi-Tomasi** (ou de Harris si spécifié) pour identifier les points d'intérêt les plus marqués dans une image en niveaux de gris.

```
corners = cv2.goodFeaturesToTrack(
    gray,
    maxCorners=100,
    qualityLevel=0.01,
    minDistance=10
)
```

Les paramètres de configuration

Les paramètres de configuration de la fonction :

- **gray** : L'image source d'entrée. Elle doit obligatoirement être au format **niveaux de gris** (8 bits ou float 32) et posséder un seul canal.
- **maxCorners=100** : La limite maximale du nombre de coins à retourner. L'algorithme sélectionnera au maximum les **100 meilleurs coins** (les plus nets). Si l'image contient moins de 100 coins respectant les critères de qualité, il retournera uniquement ceux qui sont valides.
- **qualityLevel=0.01** : Le seuil de qualité minimal pour retenir un coin, représenté par une valeur entre 0 et 1. Cette valeur est un multiplicateur appliqué au score du meilleur coin détecté dans toute l'image. Ici, tout coin ayant un score inférieur à **1 % du score du meilleur coin** sera automatiquement rejeté.
- **minDistance=10** : La distance euclidienne minimale en **pixels** allouée entre deux coins détectés. Ce paramètre garantit que l'algorithme ne va pas surcharger une même zone précise de plusieurs points rouges/cercles et permet d'obtenir une répartition spatiale homogène.
- **`minDistance=10`** : La distance euclidienne minimale en **pixels** allouée entre deux coins détectés. Ce paramètre garantit que l'algorithme ne va pas surcharger une même zone précise de plusieurs points rouges/cercles et permet d'obtenir une répartition spatiale homogène.

Format de la variable de sortie (`corners`)

La fonction retourne un tableau NumPy tridimensionnel de type `float32` contenant les coordonnées `(x, y)` des coins détectés, structuré sous la forme : `(N, 1, 2)`, où `N` est le nombre de coins trouvés.

3.4. Exemple de détection



Figura 4 : Detection de points d'intérets par Shi-Tomasi

3.5. Questions expérimentales

Faire varier :

- **maxCorners**
- **qualityLevel**
- **minDistance**

Questions

1. Quel est l'effet de qualityLevel ?
2. Quel est l'effet de minDistance ?
3. Pourquoi Shi-Tomasi est-il utilisé pour le tracking ?

4. TP 4 : FAST Features from Accelerated Segment Test

4.1. Objectif :

Utiliser **Fast** pour sélectionner les meilleurs points d'intérêt plus **rapidement**.

4.2. Fonctionnement

`keypoints = fast.detect()` est un appel de fonction qui détecte des coins dans une image en niveaux de gris à l'aide de l'algorithme FAST (Features from Accelerated Segment Test).

Fonctionnement du code.

• **gray** : L'image d'entrée convertie en niveaux de gris (requis pour l'algorithme FAST).

• **None** : Un paramètre de masque optionnel ; passer `None` signifie que l'algorithme examine l'image entière.

• **`keypoints` :** Le résultat, qui est une liste d'objets `cv2.KeyPoint` contenant les coordonnées (x, y), l'échelle et d'autres attributs des coins détectés.

4.3. Implémentation

Code : exercice 4\exo4.py

```python
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("Mosque.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

fast = cv2.FastFeatureDetector_create()

keypoints = fast.detect(gray, None)

result = cv2.drawKeypoints(
    image,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

print("Nombre de keypoints :", len(keypoints))

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
cv2.imwrite("Fast detector.jpg", result)
```

4.4. Exemple

Resultats d'execution donne :





4.5. Questions expérimentales

Vous avez sans doute remarqué le grand nombre des points détectés (plus de 7000...)
Modifier le seuil :

```
fast.setThreshold(50)
```

puis :

```
fast.setThreshold(20)
```

Question :

Analysez les résultats !!

Puis constatez : Un seuil plus faible peut détecter davantage de points, mais aussi davantage de points moins discriminants.

5. TP 5 : SIFT : Scale-Invariant Feature Transform

5.1. Objectif :

Comprendre et utiliser SIFT pour détecter et décrire des points d'intérêt.
SIFT permet notamment de rechercher des caractéristiques relativement robustes aux changements :

- d'échelle ;
- de rotation ;
- de luminosité ;

et à certaines transformations géométriques.

5.2. Interprétation

Chaque **keypoint** possède notamment :

- position
- taille
- orientation
- réponse
- octave

Le **descripteur** SIFT est généralement un vecteur de **128 valeurs**.

5.3. Implémentation

Code : exercice 5\exo5.py

Vous pouvez construire le filtre manuellement :

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

sift = cv2.SIFT_create()

keypoints, descriptors = sift.detectAndCompute(
    gray,
    None
)

print("Nombre de keypoints :", len(keypoints))
print("Taille des descripteurs :", descriptors.shape)

result = cv2.drawKeypoints(
    image,
    keypoints,
    None,
```

```
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(10,8))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

5.4. Exécution



Figure 1 : Image orginale



Figure ?? : Image aprés détection des pts d'interets avec SIFT

6. TP 6 : ORB (Oriented FAST and Rotated BRIEF)

6.1. Objectif :

Découvrir ORB et le comparer à SIFT.

ORB combine notamment une détection de type FAST avec une description binaire inspirée de BRIEF, avec une orientation pour améliorer la robustesse à la rotation.

6.2. Implémentation

Code : exercice 6\exo6.py

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

orb = cv2.ORB_create(
    nfeatures=500
)

keypoints, descriptors = orb.detectAndCompute(
    gray,
    None
)

print("Nombre de points :", len(keypoints))
print("Dimensions :", descriptors.shape)

result = cv2.drawKeypoints(
    image,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
cv2.imwrite("keypoints with ORB.jpg",result)
```

6.3. Exécution

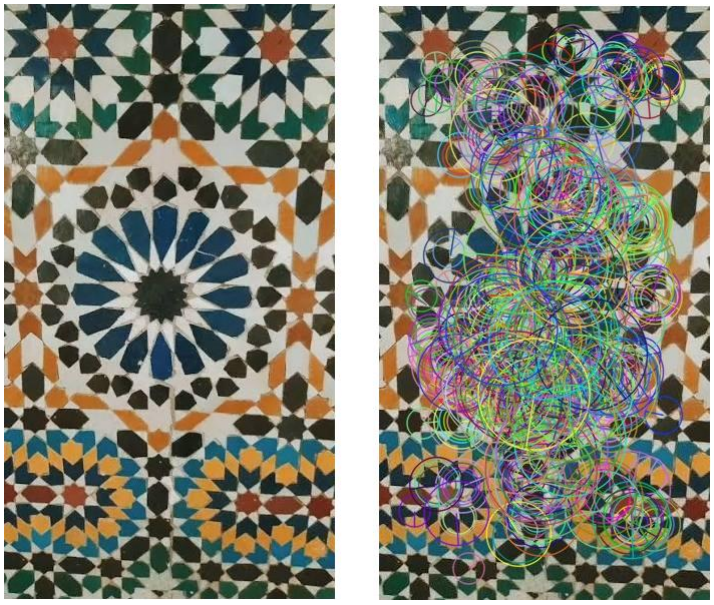


Figure 2 : ORB detection

6.4. Comparer SIFT et ORB

Selon ces critères :	SIFT	ORB
Type de descripteur	Float	Binaire
Taille typique	128	32 octets par défaut
Matching	L2	Hamming
Rapidité	Plus coûteux	Généralement plus rapide
Rotation	Oui	Oui

7. TP 7 : SURF (Speeded-Up Robust Features)

7.1. Objectif :

Comprendre et expérimenter SURF.

SURF est un descripteur local basé sur des approximations rapides du déterminant de la Hessienne et des réponses de filtres de type Haar.

7.2. Fonctionnement

OpenCV propose des fonctionnalités SURF tout comme pour SIFT. Vous initialisez un objet SURF en spécifiant des paramètres optionnels, tels que la dimension des descripteurs (64 ou 128), le mode (Upright ou Normal), etc. Tous ces détails sont clairement expliqués dans la documentation. Ensuite, tout comme avec SIFT, vous pouvez utiliser `SURF.detect()`, `SURF.compute()`, etc., pour extraire les points clés et les descripteurs.

7.3. Intallation suplementaire necessaire

Dans certaines distributions OpenCV, SURF **peut ne pas être disponible** ou peut être soumis à des différences de configuration/licence selon la version. Pour un TP moderne, il est donc intéressant de travailler sur **SIFT et ORB**, puis de traiter SURF comme comparaison historique/algorithmique lorsque l'environnement le permet.

Le dépôt **contrib** contient des fonctionnalités supplémentaires qui ne sont pas intégrées dans le cœur stable d'OpenCV, souvent parce qu'elles sont encore en cours de développement ou qu'elles concernent des cas d'usage très spécifiques.

Nous devons installer :

```
pip install opencv-contrib-python
```

Une erreur va surgir du fait que **SURF est considéré comme un algorithme "non-free" dans OpenCV**. Avec opencv-contrib-python 4.11.0, le module xfeatures2d peut être présent, mais **SURF n'est pas compilé avec OPENCV_ENABLE_NONFREE=ON**.

Il faut donc une compilation native avec Visual Studio 2022 + CMake, puis une installation directement dans votre environnement Anaconda.

7.4. Implémentation

Code : exercice 7\exo7.py

```python
import cv2

image = cv2.imread("image.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

surf = cv2.xfeatures2d.SURF_create(
    hessianThreshold=400
)

keypoints, descriptors = surf.detectAndCompute(
    gray,
    None
)

print("Keypoints :", len(keypoints))
result = cv2.drawKeypoints(
    image,
    keypoints,
    None
)

cv2.imshow("SURF", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

7.5. Travail a réaliser

Compiler opencv pour pouvoir exécuter le code !!

Déposer ensuite la solution sur le Classroom

8. Feature Matching : BFMatcher

8.1. Objectif

Associer les points d'intérêt de deux images avec BFMatcher (**Brute-Force Matcher**).

Il s'agit du constructeur de la classe **Brute-Force Matcher**. Le principe de la force brute consiste à prendre le descripteur d'un point de la première image et à calculer sa "distance" avec **tous** les descripteurs de la seconde image pour trouver le plus proche.

8.2. Implémentation

Code : exercice 8\exo8.py

```
import cv2
import matplotlib.pyplot as plt

img1 = cv2.imread("object.jpg", 0)
img2 = cv2.imread("scene.jpg", 0)

sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
bf = cv2.BFMatcher(
    cv2.NORM_L2,
    crossCheck=True
)

matches = bf.match(des1, des2)

matches = sorted(
    matches,
    key=lambda x: x.distance
)
```

```
result = cv2.drawMatches(
    img1,
    kp1,
    img2,
    kp2,
    matches[:50],
    None,
    flags=2
)

plt.figure(figsize=(15,8))
plt.imshow(result, cmap="gray")
plt.axis("off")
plt.show()
cv2.imwrite("résultat de matching.jpg", result)
```

8.3. Fonctionnement

bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True) permet d'initialiser un **objet d'appariement de caractéristiques par force brute** (**Brute-Force Matcher**). Cet outil sert à comparer et lier des points d'intérêt entre deux images différentes.

Avec la ligne de code :

```
bf = cv2.BFMatcher(
    cv2.NORM_L2,
    crossCheck=True
)

matches = bf.match(des1, des2)
```

Dont les paramètres sont :

cv2.NORM_L2 (Le type de norme)

Ce paramètre définit la méthode mathématique utilisée pour calculer la distance (la similarité) entre deux descripteurs : [1, 2]

- cv2.NORM_L2 correspond à la **distance euclidienne classique** (la racine carrée de la somme des carrés des différences).
- **Quand l'utiliser ?** C'est le choix par défaut et le plus adapté pour les descripteurs à valeurs flottantes comme **SIFT** ou **SURF**.
- ***Note :*** Si vous utilisez des descripteurs binaires (comme ORB ou BRIEF), il faudrait plutôt choisir cv2.NORM_HAMMING.

crossCheck=True (La vérification croisée)

Par défaut, ce paramètre est à False. Lorsqu'il est activé (True), il agit comme un **filtre de qualité strict** pour éliminer les faux positifs (les mauvaises correspondances).

- **Fonctionnement :** Le système valide un lien entre un point A (image 1) et un point B (image 2) **uniquement si** B est le meilleur match pour A **ET** que A est aussi le meilleur match pour B.
- **Avantage :** Cela permet d'obtenir des correspondances réciproques très cohérentes et réduit grandement le bruit (les lignes aberrantes), ce qui en fait une excellente alternative au "ratio test" de Lowe.

Comparatif des configurations de base

Descripteur utilisé	Norme recommandée	crossCheck recommandé
SIFT / SURF	`cv2.NORM_L2`	`True` (pour un tri rapide) ou `False` (si suivi d'un ratio test)
ORB / BRIEF / BRISK	`cv2.NORM_HAMMING`	`True`

8.4. Résultat

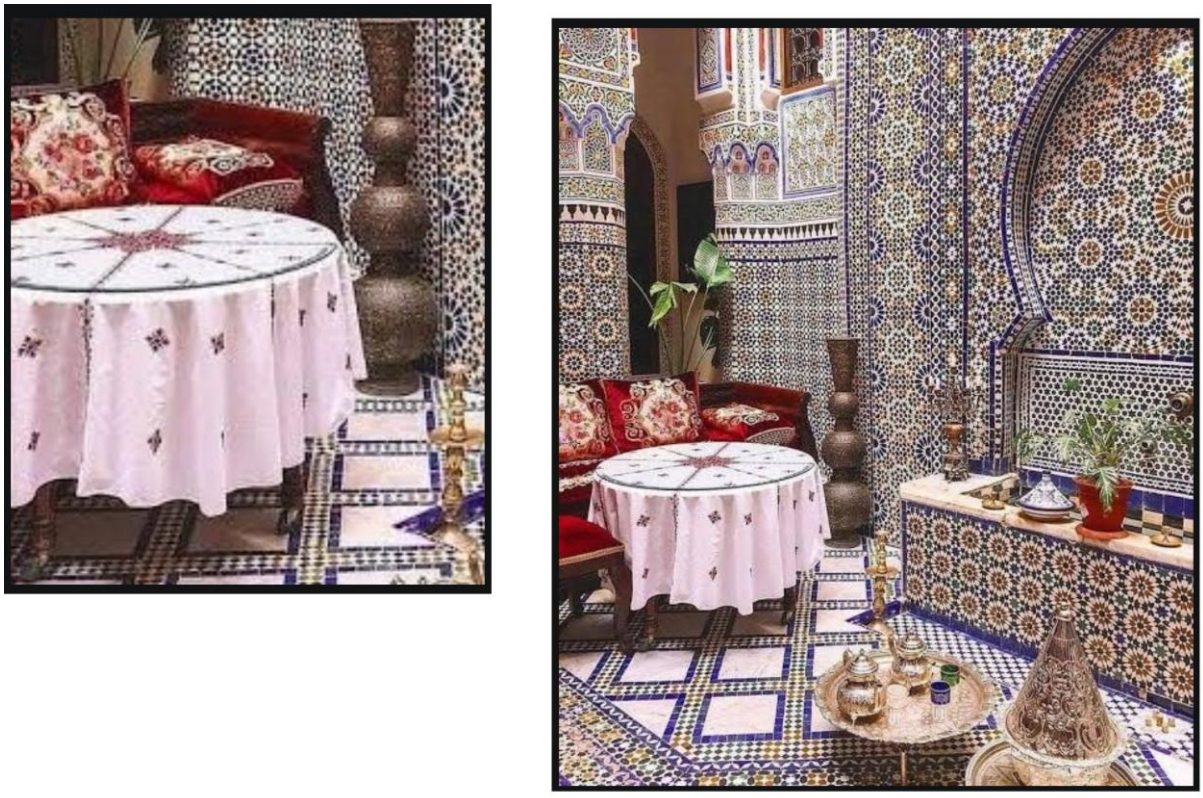


Figure 3 : Objet et scéne

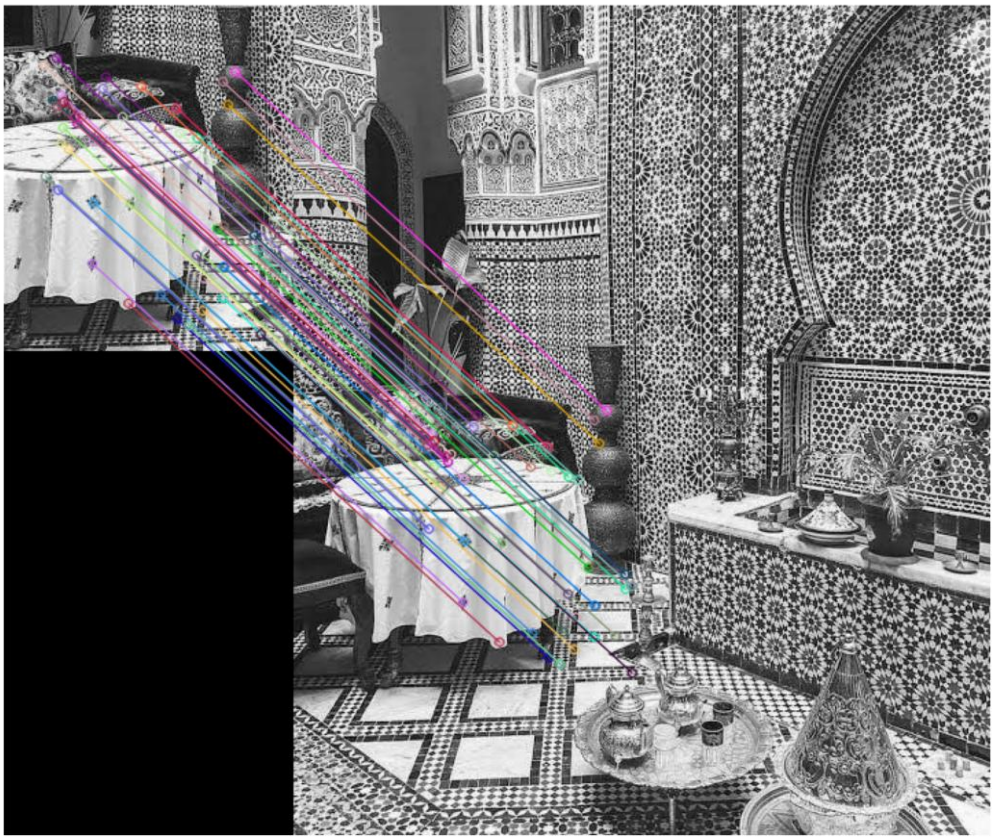


Figure 4 : résultat de correspondance

9. Homographie et RANSAC

RANSAC, qui signifie **RANdom SAmple Consensus** (Consensus d'Échantillonnage Aléatoire), est un algorithme mathématique itératif utilisé pour trouver le meilleur modèle mathématique au milieu de données très bruitées, en ignorant complètement les données aberrantes.

Contrairement aux méthodes classiques comme les moindres carrés qui tentent de faire plaisir à tout le monde (et se font fausser par les erreurs grossières), RANSAC applique une philosophie simple : **trouver le groupe de données majoritaire qui est d'accord entre lui (le consensus) et ignorer le reste**.

9.1. Objectif

Localiser un objet dans une scène à partir de correspondances SIFT.

C'est un TP particulièrement important car il fait passer de la simple détection de points à une véritable application de Computer Vision.

Image objet → SIFT → Keypoints + Descriptors → Matching → Good Matches → RANSAC → Homographie → Localisation dans la scène

9.2. Implémentation

Code : exercice 9\exo9.py

```python
import cv2
import matplotlib.pyplot as plt

img1 = cv2.imread("object.jpg", 0)
img2 = cv2.imread("scene.jpg", 0)

sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
bf = cv2.BFMatcher(
    cv2.NORM_L2,
    crossCheck=True
)

matches = bf.match(des1, des2)

matches = sorted(
    matches,
    key=lambda x: x.distance
)

result = cv2.drawMatches(
    img1,
    kp1,
    img2,
    kp2,
    matches[:50],
    None,
    flags=2
)

plt.figure(figsize=(15,8))
plt.imshow(result, cmap="gray")
plt.axis("off")
plt.show()
cv2.imwrite("résultat de matching.jpg", result)
```

9.3. Fonctionnement

bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True) permet d'initialiser un **objet d'appariement de caractéristiques par force brute** (**Brute-Force Matcher**). Cet outil sert à comparer et lier des points d'intérêt entre deux images différentes.

Avec la ligne de code :

```
bf = cv2.BFMatcher(
    cv2.NORM_L2,
    crossCheck=True
)

matches = bf.match(des1, des2)
```

Dont les paramètres sont :

cv2.NORM_L2 (Le type de norme)

Ce paramètre définit la méthode mathématique utilisée pour calculer la distance (la similarité) entre deux descripteurs : [1, 2]

- cv2.NORM_L2 correspond à la **distance euclidienne classique** (la racine carrée de la somme des carrés des différences).
- **Quand l'utiliser ?** C'est le choix par défaut et le plus adapté pour les descripteurs à valeurs flottantes comme **SIFT** ou **SURF**.
- ***Note :*** Si vous utilisez des descripteurs binaires (comme ORB ou BRIEF), il faudrait plutôt choisir cv2.NORM_HAMMING.

crossCheck=True (La vérification croisée)

Par défaut, ce paramètre est à False. Lorsqu'il est activé (True), il agit comme un **filtre de qualité strict** pour éliminer les faux positifs (les mauvaises correspondances).

- **Fonctionnement :** Le système valide un lien entre un point A (image 1) et un point B (image 2) **uniquement si** B est le meilleur match pour A **ET** que A est aussi le meilleur match pour B.
- **Avantage :** Cela permet d'obtenir des correspondances réciproques très cohérentes et réduit grandement le bruit (les lignes aberrantes), ce qui en fait une excellente alternative au "ratio test" de Lowe.

Comparatif des configurations de base

Descripteur utilisé	Norme recommandée	crossCheck recommandé
SIFT / SURF	`cv2.NORM_L2`	`True` (pour un tri rapide) ou `False` (si suivi d'un ratio test)
ORB / BRIEF / BRISK	`cv2.NORM_HAMMING`	`True`

9.4. Résultats

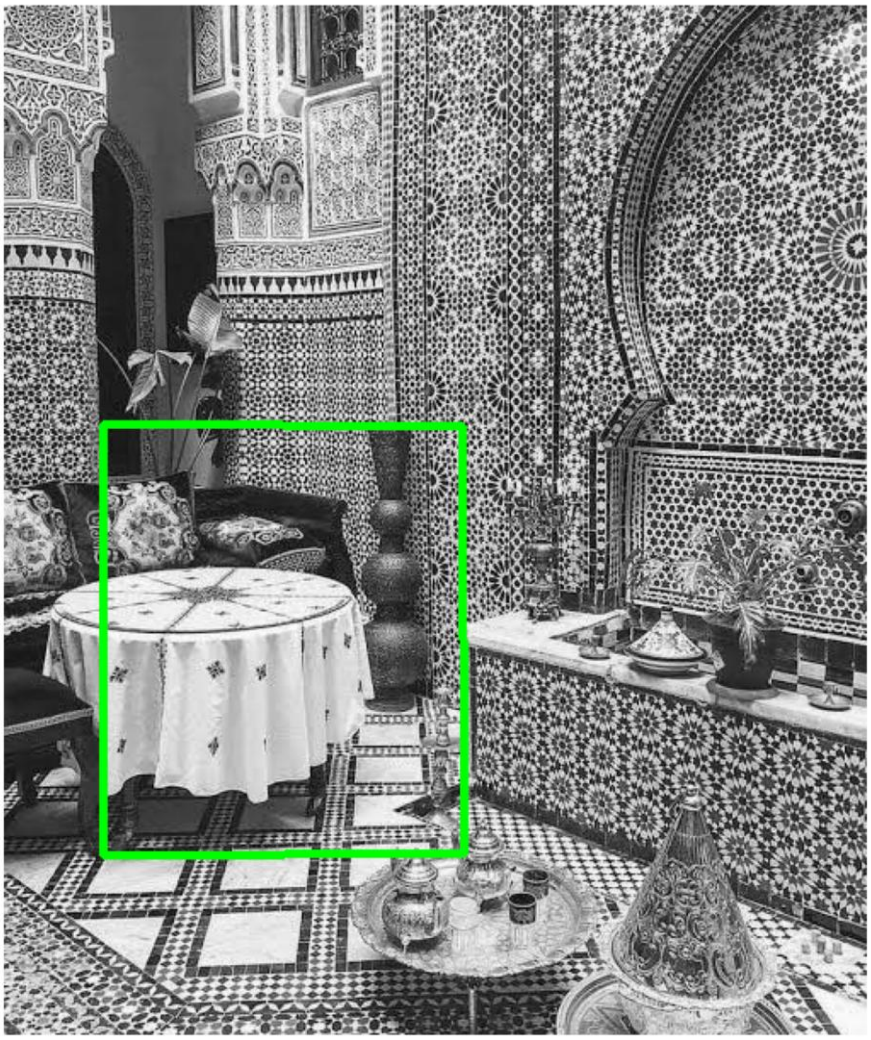


Figure 5 : Image cadrage objet

9.5. Questions Fondamentales

Questions :

1. Pourquoi faut-il au minimum 4 correspondances pour une homographie ?
2. Pourquoi utiliser RANSAC ?

3. Qu'est-ce qu'un inlier ?
4. Qu'est-ce qu'un outlier ?
5. Que se passe-t-il si les correspondances sont mauvaises ?
6. Dans quelles situations une homographie est-elle appropriée ?